

Lecture 21: See the Network

CLI Tools, Wireshark, and pyshark

EC 441 — Introduction to Computer Networking
Boston University, Spring 2026

Tuesday, April 14, 2026

Learning Objectives

By the end of this lecture you should be able to:

1. Use `ping`, `traceroute`, `ip`, `ss`, `dig`, and `tcpdump` to observe live network state and connect each output field to a concept from L13–L20.
2. Capture packets to a `.pcap` file and open them in Wireshark.
3. Navigate the Wireshark GUI to inspect protocol layers, follow TCP streams, and view the Statistics → Protocol Hierarchy.
4. Write a pyshark script to load a capture file and extract or plot fields programmatically.

1 Framing: Why Linux Network Tools?

The protocols you have studied for a semester — Ethernet, IP, TCP, UDP, DNS — do not live only in textbook diagrams. They are running right now, on the machine in front of you, producing output you can read with standard Linux tools.

Network engineers diagnose and monitor real production networks using exactly the tools covered today. Juniper JunOS (the operating system on backbone routers) runs on FreeBSD. Arista EOS (used in large data-center switches) runs on Linux. `tcpdump`, `ping`, `traceroute`, and `ss` are present on all of them. This lecture is not a simulation of professional practice — it *is* professional practice.

All demos in this lecture run inside the `ec441` Multipass virtual machine (provisioned in the previous unit). Commands shown assume an Ubuntu 24.04 guest; behavior on macOS may differ slightly.

2 Session 1: CLI Tools

2.1 `ping` — ICMP Echo

`ping` sends ICMP Echo Request messages to a destination and reports the time until an ICMP Echo Reply arrives. It is the universal first-line diagnostic for reachability and round-trip time.

Concepts connected.

- **ICMP Echo Request / Echo Reply** (L17): the two message types `ping` uses.
- **RTT estimation** (L19): the per-packet RTT values `ping` prints are exactly the signal TCP's EWMA is filtering into `srtt` and `rttvar`.
- **TTL** (L17): each reply line shows the TTL field from the incoming packet. Subtracting from the common initial values (64 for Linux/macOS, 128 for Windows) gives an estimate of the hop count.

ping demos

```
# Five pings to Google's public DNS resolver
ping -c 5 8.8.8.8

# Fast ping: send every 0.2 s -- watch RTT variance build up
ping -i 0.2 -c 20 8.8.8.8

# Large payload -- stresses Path MTU (may trigger fragmentation)
ping -c 3 -s 1400 8.8.8.8
```

What to observe. The RTT variation from packet to packet is not noise to ignore — it is the *signal* that `rttvar` tracks and that the retransmission timer (RTO) must account for. A low-variance path lets TCP set a tight RTO; a high-variance path forces a conservative one.

2.2 traceroute — Path Discovery

`traceroute` reveals the sequence of routers between you and a destination. It works by sending probe packets with TTL set to 1, then 2, then 3, and so on. Each router that receives a packet with TTL=1 decrements the field to 0, drops the packet, and returns an ICMP Time Exceeded message. That message reveals the router's IP address and the RTT to that hop.

Concepts connected.

- **TTL and hop-by-hop forwarding** (L13, L17): `traceroute` exploits TTL decrement to force each router to reveal itself.
- **ICMP Time Exceeded** (L17): the message type each router returns.
- **AS boundaries and inter-domain routing** (L16): large RTT jumps between consecutive hops often mark a crossing from one autonomous system to another, sometimes across an ocean.

traceroute demos

```
# Standard traceroute; -m 20 caps at 20 hops
traceroute -m 20 8.8.8.8

# TCP mode on port 80 -- bypasses firewalls that block ICMP probes
traceroute -T -p 80 8.8.8.8
```

What to observe.

- * * * lines indicate ICMP filtered at that hop, not a broken path — the destination may still be reachable.
- RTT jumps (e.g., 10 ms to 80 ms) usually mark AS boundary crossings or intercontinental links.
- Asymmetric routing: `traceroute` shows only the *forward* path; the return path may use entirely different routers.

2.3 ip — Interfaces, Routes, and ARP

The `ip` command is the modern replacement for the older `ifconfig` and `route` utilities. It shows and configures network interfaces, the IP routing table, and the neighbor (ARP) cache.

Concepts connected.

- **IP addressing and interfaces** (L14): `ip addr` shows each interface's IP address and prefix length.
- **Forwarding table** (L13): `ip route` shows the exact data structure a router walks to decide where to forward each packet.
- **ARP** (L08): `ip neigh` shows the ARP cache — the mapping from IP address to MAC address that allows a node to populate the Ethernet destination field.

ip demos

```
ip addr show      # interfaces, IP addresses, and prefix lengths
ip route show     # forwarding table (this is what the router walks)
ip neigh show     # ARP cache: IP -> MAC with reachability state
```

What to observe. In `ip route` output, identify the *default route* (`default via`) — any packet whose destination does not match a more-specific prefix will be forwarded to this gateway. In `ip neigh`, the state column (REACHABLE, STALE, FAILED) reflects the ARP cache entry's age and validity; STALE means the entry has not been confirmed recently but has not yet been evicted.

2.4 ss — Live TCP State

`ss` (socket statistics) reports information about open sockets. With the `-t` flag it exposes the kernel's internal TCP congestion control state for every active connection — the same variables you studied in L19 and L20.

Concepts connected.

- **TCP sockets and ports** (L18)
- **RTT estimation: `srtt` and `rttvar`** (L19)
- **Congestion window: `cwnd`, `ssthresh`** (L20)

ss demos

```
# All established TCP connections
ss -t

# Full TCP internals for each connection
ss -tipm

# Watch cwnd grow in real time during an iperf3 transfer
iperf3 -s &
iperf3 -c 127.0.0.1 -t 30 &
watch -n 0.5 'ss -tipm | grep iperf'
```

Key fields in ss -tipm output

Field	Meaning
cwnd:N	Congestion window (in MSS units)
ssthresh:N	Slow-start threshold
rtt:X/Y	Smoothed RTT (ms) / RTTVAR (ms) — the EWMA outputs from L19
retrans:N/M	Current retransmissions / total retransmissions

The `watch -n 0.5` command re-runs `ss` every 500 ms while `iperf3` is saturating the loopback interface. Watch `cwnd` grow exponentially during slow start, then linearly once it crosses `ssthresh`. If you induce loss (e.g., using `tc netem`), you will see `cwnd` halve on the next probe. This is the payoff moment for L19–L20: the congestion control state machine is no longer abstract — it is live in the kernel.

2.5 dig — DNS in Action

`dig` sends DNS queries directly and prints the full server response. It is the standard tool for diagnosing DNS resolution problems and exploring the DNS hierarchy.

Concepts connected.

- **DNS** (preview of L23): name resolution, TTLs, authoritative vs. recursive resolvers, record types.
- **UDP** (L18): DNS queries travel over UDP by default (port 53); TCP is used only for large responses (e.g., zone transfers).

dig demos

```
dig google.com           # full output: query, answer, TTL, server
dig +short google.com    # just the IP address(es)
dig @8.8.8.8 google.com  # query a specific resolver
dig +trace google.com    # follow the full resolution chain from root
dig google.com MX        # look up mail server records
```

What to observe. The TTL field in the answer section is the DNS cache lifetime: once this expires, the resolver must query the authoritative server again. `dig +trace` shows the full resolution chain — root server to TLD server to authoritative server — which will be the central topic of L23. Run `dig` while `tcpdump port 53` is running in another window to confirm that the query travels over UDP.

2.6 tcpdump — Command-Line Packet Capture

`tcpdump` captures packets directly from a network interface. It is the command-line equivalent of Wireshark and is available on virtually every Unix-like system, including router and switch operating systems.

Concepts connected. Every protocol from L08 onward is visible in `tcpdump` output: Ethernet frames, IP headers (addresses, TTL, fragmentation), TCP segments (flags, sequence numbers), UDP datagrams, ICMP messages, and DNS payloads.

tcpdump demo — capture and save

```
# Watch DNS queries in real time
sudo tcpdump -i any -n port 53

# In another terminal: trigger a DNS lookup
dig google.com

# Start a background capture to file
sudo tcpdump -i any -n -w ~/capture.pcap &

# Generate varied traffic
ping -c 5 8.8.8.8
dig google.com
curl -s https://example.com > /dev/null

# Stop the capture
sudo pkill tcpdump
```

Filter syntax. `tcpdump` uses a powerful boolean filter language:

- `host 8.8.8.8` — traffic to or from a specific IP
- `port 443` — traffic on a specific port
- `tcp, udp, icmp` — filter by protocol
- `tcp and port 80` — compound filters with `and`, `or`, `not`

The `capture.pcap` file written in this demo is the input for the Wireshark and pyshark walkthroughs in Session 2.

3 Session 2: Wireshark and pyshark

3.1 Wireshark GUI

Wireshark is a graphical packet analyzer. Transfer the capture file from the VM to your Mac:

Transfer the capture

```
# On your Mac:  
multipass transfer ec441:/home/ubuntu/capture.pcap ~/capture.pcap
```

Open the file in Wireshark. The walkthrough covers four views:

1. **Filter bar.** Type `dns` to show only DNS packets, `icmp` for ping traffic, or `tcp` for TCP segments. The display filter language is the same as `tshark` and `pyshark`.
2. **A DNS packet.** Click any DNS entry and expand the layers in the packet detail pane: Ethernet → IP → UDP → DNS. Identify the query name, the response IP, and the TTL.
3. **A TCP stream.** Select an HTTPS packet and choose *Analyze → Follow → TCP Stream*. Observe the three-way handshake: SYN from client (SEQ = 0), SYN-ACK from server (SEQ = 0, ACK = 1), and ACK from client (SEQ = 1, ACK = 1). Watch sequence numbers increment with each data segment.
4. **Statistics → Protocol Hierarchy.** This view breaks the capture into protocol layers by byte count and packet count. On a modern host, a large fraction of TCP traffic will appear under TLSv1.3 and potentially QUIC (UDP-based) — consistent with the protocol share trends discussed in L20.

The key message. Wireshark excels at deep inspection of individual conversations. When you need to ask questions across thousands of packets — “what domains did this host look up?” or “how did RTT evolve across a ten-minute transfer?” — you need a programmatic tool. That is `pyshark`.

3.2 pyshark — Packets as a Dataset

`pyshark` is a Python wrapper around `tshark`, Wireshark’s command-line back end. It loads a `.pcap` file (or attaches to a live interface) and exposes every packet as a Python object. Attribute names match Wireshark’s field names exactly: if you can see a field in the Wireshark GUI, you can access it in `pyshark`.

Key Result

The core idea: a packet capture is a *dataset*. `pyshark` lets you treat it like one — filter, iterate, aggregate, and plot — using ordinary Python.

Installation.

```
pip install pyshark # tshark must already be installed (apt install tshark)
```

3.2.1 Demo 1 — Load and Iterate

demo_pyshark_iterate_l21.py — load a pcap and print layers

```
import pyshark

cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
for pkt in cap:
    print(pkt.highest_layer, pkt.length)
```

`highest_layer` returns a string such as DNS, TCP, or TLS — the topmost protocol pyshark could decode. `length` is the total frame length in bytes.

3.2.2 Demo 2 — Inspect a Packet

demo_pyshark_inspect_l21.py — field-level access

```
cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
pkt = next(iter(cap))

print(pkt.layers)           # all protocol layers present
print(pkt.ip.src)           # source IP address
print(pkt.ip.ttl)           # TTL field (see L17)
print(pkt.transport_layer)  # 'TCP' or 'UDP'
```

The `pkt.layers` list mirrors exactly what Wireshark shows in the packet detail pane. Any field visible in Wireshark is accessible as `pkt.layer.field`.

3.2.3 Demo 3 — Extract DNS Query Names

What domains did this machine look up during the capture?

demo_pyshark_dns_l21.py — extract and deduplicate query names

```
import pyshark

cap = pyshark.FileCapture('capture.pcap', display_filter='dns')
queries = []
for pkt in cap:
    try:
        if hasattr(pkt.dns, 'qry_name'):
            queries.append(pkt.dns.qry_name)
    except AttributeError:
        pass

for q in sorted(set(queries)):
    print(q)
```

The `display_filter` argument accepts any Wireshark display filter string, exactly as you would

type it in the GUI. The `try/except` guards against packets whose DNS layer is present but incomplete (e.g., malformed responses).

What to observe. Even a brief capture of normal web browsing produces dozens of DNS queries — one for every domain loaded by a single page. Every domain name you use on the internet flows through this infrastructure. This is a preview of L23, where DNS becomes the main topic.

3.2.4 Demo 4 — Plot TCP RTT over Time

How did RTT evolve during a TCP connection?

ts shark computes the RTT for each acknowledged segment as the time delta from the data segment to its ACK. This value is stored in the field `tcp.analysis.ack_rtt` and is visible in Wireshark under *Analyze* → *Expert Information*.

demo_pyshark_rtt_l21.py — plot per-segment RTT

```
import pyshark, matplotlib.pyplot as plt

cap = pyshark.FileCapture('capture.pcap',
    display_filter='tcp.analysis.ack_rtt')
times, rtt = [], []
for pkt in cap:
    try:
        times.append(float(pkt.sniff_timestamp))
        rtt.append(float(pkt.tcp.analysis.ack_rtt) * 1000) # convert to ms
    except AttributeError:
        pass

plt.plot(times, rtt, '.', markersize=3)
plt.xlabel('Time (s)')
plt.ylabel('RTT (ms)')
plt.title('TCP RTT over time')
plt.tight_layout()
plt.savefig('rtt_plot.png')
```

Connection to L19. The resulting plot is the *raw* RTT signal — the sequence of samples that TCP’s EWMA is smoothing into `srtt`. Spikes in the plot correspond to moments of queuing delay or retransmission. The spread of points around the mean is `rttvar`. The RTO formula $RTO = srtt + 4 \cdot rttvar$ is designed to sit above nearly all the points in this plot without being unnecessarily large.

Backup capture. If the live class capture does not contain enough TCP RTT data (e.g., all traffic is QUIC or TLS), generate a clean capture using iperf3 on the loopback interface:

```
iperf3 -s &
sudo tcpdump -i lo -w /home/ubuntu/iperf_capture.pcap &
iperf3 -c 127.0.0.1 -t 10
sudo pkill tcpdump
```


The loopback interface carries unencrypted TCP traffic with fully visible headers and tshark-computable ACK RTTs.

4 Summary

Tool	What it shows	Concepts
<code>ping</code>	ICMP RTT, TTL	L17 (ICMP), L19 (RTT)
<code>tracert</code>	Per-hop path and latency	L13 (forwarding), L17 (ICMP TTL)
<code>ip</code>	Interfaces, routes, ARP	L08 (ARP), L13 (forwarding), L14 (addressing)
<code>ss</code>	Live TCP cwnd, RTT	L19 (EWMA), L20 (congestion control)
<code>dig</code>	DNS queries and records	L18 (UDP), L23 preview (DNS)
<code>tcpdump</code>	Raw packet capture	All protocols
Wireshark	GUI protocol inspection	All protocols
pyshark	Programmatic pcap analysis	All protocols

The tools are not new content. They are a lens through which the content from the entire semester becomes observable and measurable.

References

- `man ping`, `man tracert`, `man ip`, `man ss`, `man dig`, `man tcpdump` — the authoritative reference for each tool’s flags and output format.
- pyshark on GitHub — documentation and examples for the Python wrapper used in Session 2.
- Wireshark display filter reference — complete list of field names usable in `display_filter` strings and pyshark attribute access.
- Cloudflare Radar — live data on protocol distribution (TCP vs. QUIC, HTTP versions); useful context for the Protocol Hierarchy view in Wireshark.